BiasAperture — Research Sprint: Low-Level Technical Specification

Target Audience:	Core Developers, Mathematical Implementers, Statistical Auditors
Document Level:	Low-Level (Mathematical Formulations, Exact Algorithms, Tensor Slicing & Code Specs)
Date:	August 2026
Context:	Milestone M1 Completion & 20-Track Parallel Research Sprint

1 Mathematical Fairness Formulations & Backend Harmonization

Let $Y \in \{0,1\}$ denote the binary ground-truth label, $\hat{Y} \in \{0,1\}$ denote the model prediction, and $A \in \{a_1, a_2, \ldots, a_K\}$ denote the protected demographic attribute across K mutually exclusive subgroups.

Metric Name	**Mathematical Definition**	**Fair Value / Range**
Demographic Parity Difference	$\max_a P(\hat{Y}=1 \vert A=a) - \min_a P(\hat{Y}=1 \vert A=a)$	0.0 $[0,1]$
Equalized Odds Difference	$\max\big(\max_a \mathrm{TPR}_a - \min_a \mathrm{TPR}_a,\ \max_a \mathrm{FPR}_a - \min_a \mathrm{FPR}_a\big)$	0.0 $[0,1]$
Equal Opportunity Difference	$\max_a \mathrm{TPR}_a - \min_a \mathrm{TPR}_a$	0.0 $[0,1]$
Disparate Impact Ratio	$\min_a P(\hat{Y}=1 \vert A=a) / \max_a P(\hat{Y}=1 \vert A=a)$	1.0 $[0,1]$

1.1 Demographic Parity Difference (DPD)

Measures the spread in selection rates across demographic groups unconditional on ground truth:

$$\mathrm{DPD} = \max_{a \in A} P(\hat{Y} = 1 \mid A = a) - \min_{a \in A} P(\hat{Y} = 1 \mid A = a)$$

- **Fairlearn API**: `fairlearn.metrics.demographic_parity_difference(y_true, y_pred, sensitive_features=A`
- **AIF360 API**: Evaluated via `BinaryLabelDatasetMetric.statistical_parity_difference()`.

1.2 Equalized Odds Difference (EOD) — Resolving Definitional Divergence

Let $\mathrm{TPR}_a = P(\hat{Y} = 1 \mid Y = 1, A = a)$ and $\mathrm{FPR}_a = P(\hat{Y} = 1 \mid Y = 0, A = a)$.

The Divergence Found in Research (Tracks 09, 10, 14)

- **Fairlearn**: Implements the **worst-case gap** (Hardt et al., 2016):

$$\mathrm{EOD}_{\mathrm{Fairlearn}} = \max\left(\max_a \mathrm{TPR}_a - \min_a \mathrm{TPR}_a,\ \max_a \mathrm{FPR}_a - \min_a \mathrm{FPR}_a\right)$$

- **AIF360 Native**: Implements the **average gap**:

$$\mathrm{EOD}_{\mathrm{AIF360,\ native}} = \frac{1}{2}\left(|\mathrm{TPR}_u - \mathrm{TPR}_p| + |\mathrm{FPR}_u - \mathrm{FPR}_p|\right)$$

The Locked Resolution

Reporting AIF360's native mean would cause a false backend divergence. Therefore, `AIF360Backend` **must bypass** `average_odds_difference` and compute the max-of-gaps directly from raw TPR and FPR primitives:

```
# Harmonized EOD in AIF360Backend
tpr_gap = max(group_tprs.values()) - min(group_tprs.values())
fpr_gap = max(group_fprs.values()) - min(group_fprs.values())
eod_value = max(tpr_gap, fpr_gap)
```

1.3 Equal Opportunity Difference (EOP) — Resolving Sign Mismatch

$$\mathrm{EOP} = \max_{a\in A} \mathrm{TPR}_a - \min_{a\in A} \mathrm{TPR}_a$$

The Divergence Found in Research (Track 14)

- **Fairlearn**: Returns unsigned difference ≥ 0.
- **AIF360 Native**: Returns signed difference $\mathrm{TPR}_u - \mathrm{TPR}_p \in [-1, 1]$.

The Locked Resolution

`AIF360Backend` applies `abs()` before storing to `MetricResult.metric_value` to prevent signed outputs (e.g. -0.293 vs $+0.293$) from tripping divergence alerts:

```
eop_value = abs(float(metric.equal_opportunity_difference()))
```

1.4 Disparate Impact Ratio (DIR) & Edge Cases

$$\mathrm{DIR} = \frac{\min_{a\in A} P(\hat{Y} = 1 \mid A = a)}{\max_{a\in A} P(\hat{Y} = 1 \mid A = a)}$$

- **Symmetric Bounded Form**: Scaled to $[0, 1]$, where 1.0 represents demographic parity.
- **Zero-Denominator Edge Handling**:
  - If $\max_a P(\hat{Y} = 1 \mid A = a) = 0.0$ (no positive predictions in any group), $\mathrm{DIR} = 1.0$ (no disparate selection).
  - If $\min_a P(\hat{Y} = 1 \mid A = a) = 0.0$ and $\max > 0$, $\mathrm{DIR} = 0.0$ (complete disparate exclusion).
- **Four-Fifths Rule Context**: Evaluated relative to the legal 0.80 threshold via bootstrap confidence intervals (whether the 95% CI interval sits entirely below, straddles, or exceeds 0.80).

2 Statistical Engine Implementation Details

2.1 Vectorized BCa Bootstrap Confidence Intervals

`scipy.stats.bootstrap` cannot compute BCa intervals on multi-group statistics. BiasAperture implements a standalone vectorized BCa/percentile engine on `numpy.random.Generator`.

```
+---------------------------------------------+
|      Original Data (N subjects)             |
+----------------------+----------------------+
                       |
                       v
+---------------------------------------------+
|   Stratified Index Resampling (B=1000)      |
|   rng.choice(idx_g, size=len(idx_g))        |
+----------------------+----------------------+
                       |
                       v
+---------------------------------------------+
|    Metric Evaluation Loop (B replicates)    |
|    theta_hat_b = [f(boot_b) for b in B]     |
+----------------------+----------------------+
                       |
                       v
+---------------------------------------------+
|       Jackknife Acceleration (a) &          |
|         Bias Correction (z0)                |
+----------------------+----------------------+
                       |
      +------------------+------------------+
      v                                     v
[ Is BCa Valid? ]                     [ Acceleration / ]
+-- YES --> Compute BCa Limits         [ Bias Degenerate?]
+-- NO  --> Percentile Fallback  ---->  Percentile Fallback
```

Mathematical Steps for BCa

1. **Bootstrap Replication**: Compute $\hat{\theta}_1^*, \ldots, \hat{\theta}_B^*$ over stratified resamples.
2. **Bias-Correction Parameter** (z_0):

$$z_0 = \Phi^{-1}\left(\frac{1}{B}\sum_{b=1}^{B}\mathbb{I}(\hat{\theta}_b^* < \hat{\theta})\right)$$

3. **Jackknife Acceleration** (a): Using leave-one-out jackknife estimates $\hat{\theta}_{(i)}$ for $i = 1, \ldots, n$:

$$\bar{\theta}_{(\cdot)} = \frac{1}{n}\sum_{i=1}^{n}\hat{\theta}_{(i)}, \quad a = \frac{\sum_{i=1}^{n}(\bar{\theta}_{(\cdot)} - \hat{\theta}_{(i)})^3}{6\left[\sum_{i=1}^{n}(\bar{\theta}_{(\cdot)} - \hat{\theta}_{(i)})^2\right]^{3/2}}$$

4. **BCa Adjusted Percentiles** (α_1, α_2):

$$\alpha_1 = \Phi\left(z_0 + \frac{z_0 + z_{\alpha/2}}{1 - a(z_0 + z_{\alpha/2})}\right), \quad \alpha_2 = \Phi\left(z_0 + \frac{z_0 + z_{1-\alpha/2}}{1 - a(z_0 + z_{1-\alpha/2})}\right)$$

5. **Fallback Condition**: If $|a| > 0.5$, z_0 is undefined, or the adjusted quantiles fall outside $[0, 1]$, fall back immediately to standard empirical percentiles:

$$\text{CI}_{\text{fallback}} = \left[\text{Percentile}\left(\hat{\theta}^*, 2.5\right), \text{ Percentile}\left(\hat{\theta}^*, 97.5\right)\right]$$

2.2 Chi-Squared Contingency & Holm-Bonferroni Adjustment

1. **Table Layout**: For attribute A with K subgroups, construct $2 \times K$ matrix:

$$O = \begin{bmatrix} n_{1,\text{pos}} & n_{2,\text{pos}} & \cdots & n_{K,\text{pos}} \\ n_{1,\text{neg}} & n_{2,\text{neg}} & \cdots & n_{K,\text{neg}} \end{bmatrix}$$

2. **Independence Test**: Compute $\chi^2 = \sum \frac{(O_{ij}-E_{ij})^2}{E_{ij}}$ with degrees of freedom dof $= K - 1$.
3. **Holm-Bonferroni Step-Down FWER Adjustment**: Given M sorted p-values $p_{(1)} \leq p_{(2)} \leq \cdots \leq p_{(M)}$:

$$p_{(k)}^{\text{adj}} = \min\left(1, \max_{j \leq k}\left[(M - j + 1) \cdot p_{(j)}\right]\right)$$

Reject the null hypothesis if $p_{(k)}^{\text{adj}} < \alpha = 0.05$.

3 FairFace Architecture & Preprocessing Deep Dive

3.1 ResNet-34 Multi-Task Head & Tensor Slicing

The baseline model is an ImageNet-pretrained ResNet-34 backbone terminating in a single fully connected layer of 18 output units:

```
[ Input Image 3x224x224 ]
          |
          v
[ ResNet-34 Backbone (Layers 1-4) ]
          |
          v
[ AdaptiveAvgPool2d -> 512-d ]
          |
          v
[ Linear(in_features=512, out_features=18) ]
          |
          +-- Slice [0:7]   --> Softmax --> Race Probabilities (7 Classes)
          +-- Slice [7:9]   --> Softmax --> Gender Probabilities (2 Classes)
          +-- Slice [9:18]  --> Softmax --> Age Probabilities (9 Classes)
```

```python
# Exact PyTorch slicing from predict.py
outputs = model(images) # Shape: (batch_size, 18)

race_logits = outputs[:, 0:7] # 7 race classes
gender_logits = outputs[:, 7:9] # 2 gender classes
age_logits = outputs[:, 9:18] # 9 age bins

race_preds = torch.argmax(race_logits, dim=1)
gender_preds = torch.argmax(gender_logits, dim=1)
age_preds = torch.argmax(age_logits, dim=1)
```

3.2 Preprocessing & Alignment Pipeline

1. **Face Detection & Alignment**:
   - `dlib.cnn_face_detection_model_v1` detects bounding box.
   - `dlib.shape_predictor_5_face_landmarks` identifies 5 facial landmarks (two eye corners, nose base).
   - `dlib.get_face_chips(img, shapes, size=300, padding=0.25)` extracts aligned 300×300 square crop.
2. **Torchvision Tensor Normalization**:
   - Resize to 224×224.
   - Scale pixel values to $[0.0, 1.0]$.
   - Normalize with ImageNet constants:

$$\mu = [0.485, 0.456, 0.406], \quad \sigma = [0.229, 0.224, 0.225]$$

4 Resolution Guide for the 21 Cross-Track Conflicts

#	Discrepancy / Conflict	Tracks	Technical Resolution Applied in Code
1	Equalized Odds: max-gap vs mean-gap	09, 10, 17	`AIF360Backend` manually computes $\max(\Delta\text{TPR}, \Delta\text{FPR})$ using raw TPR/FPR primitives.
2	Disparate Impact Ratio formulation	09, 10, 13	Symmetric bounded ratio $\min / \max \in [0, 1]$ adopted as headline metric; pairwise matrix retained for diagnosis.
3	Signed vs Unsigned EOP	14	`AIF360Backend` calls `abs()` on native output before populating `MetricResult`.
4	Cross-group metric row shape	11, 13, 14	Cross-group metrics output summary rows with explicit `subgroup="ALL"` and per-group deviation tables.
5	Timing of $n \geq 30$ sample size guard	09, 14	Input arrays are pre-filtered in `fairness/base.py` *before* invoking backend libraries.
6	Zero-denominator DIR handling	13	Evaluated conditionally: $0/0 \to 1.0$, $x/0 \to 0.0$ (no new schema field needed).
7	EOD two-stratum p-value combination	12, 14	Report the conservative (higher) p-value: $p = \max(p_{\text{TPR}}, p_{\text{FPR}})$.
8	Multiple-testing correction family	12	Holm-Bonferroni correction applied per protected demographic attribute family.
9	AIF360 privileged/unprivileged leakage	10	Report layer maps all labels to objective strings: `subgroup` and `reference_group`.
10	Regulatory-tag storage mechanism	08, 19	Static lookup dictionary in `bias_aperture.report` mapping metric names to Article 10 / NIST clauses (leaves `schema.py` unmodified).
11	EU AI Act Art. 10(5) ownership	08	Embedded directly in the Model Card / Datasheet governance section in `report/`.

#	Discrepancy / Conflict	Tracks	Technical Resolution Applied in Code
12	`ExplanationResult` schema integration	15, 16	Maintained as an internal dataclass in `explainability.py` without modifying M1 `schema.py`.
13	Dashboard pass/fail semantics	05	Framed as an analytical "scanning aid" rather than a legal certification verdict.
14	SHAP visual format	05, 15	Rendered strictly as base64-encoded inline raster PNGs (`data:image/png;base64,...`).
15	Checkpoint filename mismatch	01, 04	Documented `fairface_alldata_20191111.pt` as primary default and `res34_fair_align_multi_7_20190809.pt` as alternative.
16	Dataset size (108,501 vs 97,698)	01, 03	Verified 97,698 released images on disk (86,744 train + 10,954 val); 108,501 documented as pre-discard total.
17	Preprocessing method (MTCNN vs dlib)	04, 07	Stale MTCNN references updated to `dlib` CNN face detector with 5-point alignment.
18	File path column ambiguity	01	`PredictionsFileInterface` maps `face_name_align` for predictions and `file` for raw label CSVs.
19	Track prompt label drift	17, 18	Resolved in master project register (Track 17 = Strategy Pattern, Track 18 = pytest Suite).
20	Ingestion sample check naming	03	Named `insufficient_sample_at_ingestion` to avoid colliding with `MetricResult.insufficient_sample`.
21	Constant duplication in statistics	11	`fairness/statistics.py` imports `MIN_SUBGROUP_SAMPLE_SIZE` directly from `schema.py`.